

快速上手教程

本教程面向第一次使用 `opsx-dev-pipeline` 的开发者，以 **Claude Code + Backend Stack** 为例，带你从环境准备走到首次变更交付。

1. 完成本教程后，你将获得什么

完成本教程后，你会在自己的项目中：

- 初始化 OpenSpec 和 Backend Schema；
- 安装 Claude Code 使用的 Pipeline Skill、Commands 与 Hooks；
- 了解 Pipeline、Phase、Route、Stack、Tool 和 Manifest 等核心概念；
- 完成一次“探索 → 提案 → 实施 → 归档 → 提交与推送”的 Backend 变更；
- 学会在会话中断后恢复 Pipeline。

`opsx-dev-pipeline` 应安装到你要开发的消费方项目中。不要在 `opsx-dev-pipeline` CLI 自身的源码仓库中运行 `init`。

2. 开始前准备

2.1 环境要求

请先准备：

工具	要求	用途
Node.js	20 或更高版本	运行 CLI、OpenSpec 和 Pipeline 脚本
Git	可执行 <code>git</code> 命令	管理分支、提交与交付
OpenSpec CLI	1.6.0 或更高版本	管理 proposal、spec、tasks 和 archive
Claude Code	能在目标项目中正常启动	调用生成的 Commands 和 Skills

检查本机环境：

```
1 node --version
2 git --version
3 openspec --version
4 git config user.name
5 git config user.email
```

如果尚未安装 OpenSpec，请执行：

```
1 npm install -g @fission-ai/openspec@latest
2 openspec --version
```

初始化前，`opsx-dev-pipeline` 会执行 Preflight。OpenSpec 不在 `PATH` 中或版本低于 1.6.0 时，命令会直接停止，不会写入部分文件。

2.2 准备目标项目

进入你要接入 Pipeline 的 Git 项目：

```
1 | cd /path/to/your-backend-project
2 | git status
```

建议先确认：

1. 当前目录是正确的项目根目录；
2. 项目已经执行过 `git init`；
3. 现有改动已经提交，或至少已备份并清楚其用途；
4. Git 的 `user.name` 和 `user.email` 已配置；
5. 如需推送，项目已配置 Git remote。

可用以下命令检查 remote：

```
1 | git remote -v
```

没有 remote 也可以完成本地初始化和开发；交付时选择 `local-only` 即可只创建本地提交。

3. 初始化项目

3.1 先预览安装计划

首次使用时，建议先执行 Dry Run：

```
1 | npx opsx-dev-pipeline@latest init \
2 |   --tool claude \
3 |   --stack backend \
4 |   --yes \
5 |   --dry-run
```

这个命令会：

- 检查 OpenSpec CLI 和版本；
- 生成完整安装计划；
- 展示将创建或处理的文件；
- 不会调用 `openspec init`，也不会写入文件。

参数含义：

参数	含义
<code>--tool claude</code>	Tool 使用 Claude Code
<code>--stack backend</code>	安装 Backend Schema 和规则
<code>--yes</code>	非交互执行；遇到冲突文件时默认跳过
<code>--dry-run</code>	只预览，不写文件

`--yes` 不等于覆盖。只有 `--force` 才会覆盖冲突文件。已有项目不要为了省事直接使用 `--force`，应先查看差异并确认哪些内容可以覆盖。

3.2 正式初始化

确认 Dry Run 结果符合预期后，执行：

```
1 npx opsx-dev-pipeline@latest init \  
2   --tool claude \  
3   --stack backend \  
4   --yes
```

如果希望逐项选择 Tool、Stack、文档语言和其他选项，可使用交互模式：

```
1 npx opsx-dev-pipeline@latest init
```

Backend 还支持通过 `--tech-stack` 选择更具体的技术栈。例如：

```
1 # Java + Spring Boot  
2 npx opsx-dev-pipeline@latest init \  
3   --tool claude \  
4   --stack backend \  
5   --tech-stack java-spring-boot \  
6   --yes  
7  
8 # Python + FastAPI  
9 npx opsx-dev-pipeline@latest init \  
10  --tool claude \  
11  --stack backend \  
12  --tech-stack python-fastapi \  
13  --yes
```

默认文档语言为中文。需要英文产出时增加 `--lang en`。

4. 检查初始化结果

初始化完成后，先运行只读诊断：

```
1 npx opsx-dev-pipeline@latest doctor
```

然后检查项目中的关键产物：

```
1  | .claude/
2  |   ├── skills/opsx-dev-pipeline/      # Phase 0-7 Pipeline Skill
3  |   └── commands/opsx/                 # Claude Code 斜杠命令
4
5  | openspec/
6  |   ├── config.yaml                   # 项目上下文、规则和 Route 配置
7  |   └── schemas/backend/              # Backend proposal / design / spec / tasks 模板
8
9  | CLAUDE.md                           # Claude Code 项目指令
10 | package.json 或 opsx-dev-pipeline.json # opsxDevPipeline Manifest
```

这些术语的含义是：

- **Tool**：AI 宿主，本教程选择 `claude`；
- **Stack**：项目大类，本教程选择 `backend`；
- **Manifest**：托管清单；项目已有 `package.json` 时存放在 `package.json#opsxDevPipeline`，否则存放在根目录 `opsx-dev-pipeline.json`；
- **Asset**：CLI 创建并由 `sync` / `upgrade` 管理的单个文件；
- **Pipeline**：从变更提案到 Git 交付的端到端流程；
- **Phase**：Pipeline 中编号为 0-7 的阶段；
- **Route**：根据风险决定实际执行哪些 Phase 的分级。

如果 Commands 没有立即出现在 Claude Code 中，请关闭并重新打开当前项目会话，让宿主重新加载 `.claude/commands/` 和 `.claude/skills/`。

5. 完成首次 Backend Pipeline

下面用一个低风险示例熟悉完整流程：

为健康检查接口增加应用版本字段，并保持现有状态码和响应字段不变。

开始前建议从主分支创建独立 feature 分支：

```
1 | git switch -c feature/health-version
```

5.1 先探索需求

在 Claude Code 中输入：

```
1 | /opsx:explore "为健康检查接口增加应用版本字段，并保持现有状态码和响应字段不变"
```

Explore 会分析现有实现、调用关系、测试和潜在风险，但不会创建正式 change，也不会修改代码。请重点确认：

- 实际健康检查入口在哪里；

- 是否已有可复用的版本信息来源；
- 哪些响应字段必须保持兼容；
- 应增加哪些测试或验证步骤。

5.2 启动正式 Pipeline

确认探索结论后输入：

```
1 /opsx:dev-pipeline "基于刚才的探索结果，为健康检查接口增加应用版本字段，并保持现有状态码和响应字段不变"
```

Pipeline 会按 Phase 推进，并在关键决策点等待你的确认。不要把“继续”理解为授权后续所有操作；每个关键动作仍会单独确认。

5.3 Phase 0：确认入口与 Route

Phase 0 会执行 Preflight、确认是否关联外部需求，并评估 Route：

Route	适合场景	默认执行的 Phase
trivial	typo、格式化、注释等无行为变化的修改	0、2、6
standard	功能开发、Bug 修复、重构	0、1、2、5、6
full	核心逻辑、数据库 schema、安全相关修改	0-7

本示例通常可使用 `standard`，但 Route 必须根据项目实际影响判断，并由你确认。执行中发现风险升高时可以升级 Route；Route 只能向上升级，不能降级。

5.4 Phase 1：审阅提案

`standard` 和 `full` Route 会生成 proposal、specs、design 和 tasks 等制品。确认前至少检查：

- 目标和非目标是否清楚；
- 是否明确保持现有接口兼容；
- 版本信息来源是否合理；
- 异常和边界场景是否被覆盖；
- tasks 是否包含实现与验证。

方向不对时选择修改提案，不要为了尽快进入编码而直接批准。

5.5 Phase 2：实施并确认结果

提案批准后，Agent 会按 tasks 实施代码和测试。实施结束时检查：

- 修改范围是否与 proposal 一致；
- 是否复用了项目已有能力；

- 是否真实运行了对应验证命令；
- 测试输出是否明确通过；
- 未跟踪文件中是否混入临时文件或敏感信息。

确认实施结果后，Pipeline 才会进入后续 Phase。

5.6 审查、测试与归档

- `full` Route 会进入 Phase 3 代码审查和 Phase 4 单测门禁；
- `standard` Route 默认跳过 Phase 3、4，但仍会进入 Phase 5；
- Phase 5 会执行 verify、询问是否同步 Delta Specs，并在确认后归档 change。

跳过审查或测试不代表可以忽略验证。即使使用 `standard` Route，也应确认项目已有测试、构建或接口检查真实通过。

5.7 选择交付方式

归档后需要显式选择后续操作：

选择	行为	新手建议
<code>push-only</code>	Phase 6 commit 并推送源分支，不合并	首次使用推荐，方便在 PR 中继续审阅
<code>merge</code>	Phase 6 推送源分支后进入 Phase 7 合并交付	熟悉流程并确认团队合并规则后再使用
终止流程	暂停在归档完成状态，不继续 Git 交付	暂时不准备提交或推送时使用

本示例建议选择 `push-only`。Phase 6 会分别询问 commit 和源分支 push，二者不是一次授权。选择 `merge` 时才会进入 Phase 7，并再次确认合并、目标分支推送、清理和标签等操作。

如果项目没有 remote，Phase 6 会询问配置远程、转为 `local-only` 或终止。`local-only` 是无远程时的降级路径，只创建本地 commit，不是 Phase 5 的常规交付选项。

6. 如何判断首次运行成功

不要只依据 Agent 的口头总结。请核对以下证据：

1. Pipeline 最终摘要显示 change 名称、Route、测试或 verify 结果、归档路径和交付模式；
2. `openspec/changes/archive/` 中存在本次归档结果；
3. Git 中存在本次实现 commit 和最终 Pipeline 状态 commit；
4. 选择 `push-only` 时，源分支已推送到 remote；
5. `git status` 没有意外遗留的业务文件或敏感文件。

可手动检查：

```
1 git status
2 git log --oneline -5
3 git branch --show-current
4 git remote -v
```

如果选择了 `push-only`，还可以到代码托管平台确认远程源分支和提交是否存在。

7. 暂停与继续

Pipeline 状态会保存在：

```
1 openspec/.pipeline-state/<change-name>.json
```

会话中断、机器重启或主动暂停后，在项目根目录重新打开 Claude Code，输入：

```
1 /opsx:dev-pipeline <change-name>
```

Phase 0 会核对状态文件、OpenSpec change、Git 分支和任务事实，再从可信的断点恢复。

不要直接编辑 `.pipeline-state/*.json`。状态必须通过 Pipeline 内置脚本更新，否则可能破坏门禁和恢复逻辑。

8. 使用其他 Tool

初始化时替换 `--tool` 即可。功能相同，但命令语法和 Asset 目录不同：

Tool	初始化参数	Pipeline 入口	主要目录
Claude Code	<code>--tool claude</code>	<code>/opsx:dev-pipeline</code>	<code>.claude/skills/</code> 、 <code>.claude/commands/</code>
Cursor	<code>--tool cursor</code>	<code>/opsx-dev-pipeline</code>	<code>.cursor/rules/</code> 、 <code>.cursor/commands/</code>
Codex	<code>--tool codex</code>	<code>\$opsx-dev-pipeline</code>	<code>.agents/skills/</code>
OpenCode	<code>--tool opencode</code>	<code>/opsx:dev-pipeline</code>	<code>.opencode/skills/</code> 、 <code>.opencode/commands/</code>

例如使用 Cursor 初始化 Backend Stack：

```
1 npx opsx-dev-pipeline@latest init \
2   --tool cursor \
3   --stack backend \
4   --yes
```

同一项目可以安装多个 Tool。`sync` 和 `upgrade` 会根据 Manifest 更新已安装 Tool 的托管 Asset。

9. 命令速查

`opsx-dev-pipeline` 包含两类命令：

- 系统命令：在 Claude Code 对话框中输入，格式为 `/opsx:xxx`，用于需求探索和研发流程；

- **工程命令：**在终端中执行，格式为 `npx opsx-dev-pipeline@latest xxx`，用于安装和维护 Pipeline Asset。

9.1 系统支持的 `/opsx:xxx` 命令

以下命令在完成 Claude Code Tool 初始化后使用：

命令	作用	常用示例
<code>/opsx:dev-pipeline</code>	启动或恢复 Phase 0-7 Pipeline，根据 Route 编排提案、实施、验证、归档和 Git 交付	<code>/opsx:dev-pipeline "新增健康检查接口"</code>
<code>/opsx:explore</code>	只读探索需求、代码、影响范围和可选方案，不创建正式 change	<code>/opsx:explore "分析健康检查接口的实现"</code>
<code>/opsx:propose</code>	创建 OpenSpec change，生成 proposal、specs、design 和 tasks	<code>/opsx:propose "新增健康检查接口"</code>
<code>/opsx:apply</code>	按 change 的 tasks 实施代码，并更新任务和 Pipeline 状态	<code>/opsx:apply add-health-check</code>
<code>/opsx:verify</code>	验证实现与 proposal、specs、design 和 tasks 是否一致	<code>/opsx:verify add-health-check</code>
<code>/opsx:sync</code>	将当前 change 的 Delta Specs 同步到主 Specs	<code>/opsx:sync add-health-check</code>
<code>/opsx:archive</code>	验证并归档 OpenSpec change，记录后续交付选择	<code>/opsx:archive add-health-check</code>
<code>/opsx:init</code>	分析当前项目，初始化或更新 <code>openspec/config.yaml</code> 中的项目上下文和规则	<code>/opsx:init</code>
<code>/opsx:dev-spec-design</code>	生成系统分析与设计说明书（精简版）	<code>/opsx:dev-spec-design "设计订单查询能力"</code>
<code>/opsx:grill-me</code>	通过逐题访谈完善计划或设计，暴露遗漏的约束和风险	<code>/opsx:grill-me</code>
<code>/opsx:grilling</code>	对计划、决策或想法进行更严格的质询	<code>/opsx:grilling</code>

推荐的新功能主路径是：

```
1 # 第一步：需求探索
2 /opsx:explore "需求描述"
3
4 # 第二步（可选）：思路拷问
5 /opsx:grill-me
6
7 # 第三步：启动完整流水线（Phase 0 会推荐 Route，通常 standard 或 full）
8 /opsx:dev-pipeline "基于探索结果完成该需求"
```

需要手动控制每一步时，可以使用：

```
1 /opsx:propose "需求描述"
2 /opsx:apply <change-name>
3 /opsx:verify <change-name>
4 /opsx:sync <change-name>
5 /opsx:archive <change-name>
```

9.2 工程命令 `npx opsx-dev-pipeline@latest xxx`

以下命令在项目终端中执行：

命令	作用	常用选项或示例
<code>npx opsx-dev-pipeline@latest init</code>	初始化 OpenSpec，并安装所选 Tool、Stack、Skills、Commands、Hooks 和 Manifest	<code>--tool claude --stack backend --yes --dry-run</code>
<code>npx opsx-dev-pipeline@latest doctor</code>	只读检查 Manifest、模板版本和 Stack 配置，不修改文件	<code>--json</code> 、 <code>--stack</code> 、 <code>--dir <path></code>
<code>npx opsx-dev-pipeline@latest list-tools</code>	列出支持的 Tool adapter 及其能力	<code>--json</code>
<code>npx opsx-dev-pipeline@latest sync</code>	根据 Manifest 重新渲染当前已经托管的 Asset	<code>--dry-run</code> 、 <code>--yes</code> 、 <code>--force</code> 、 <code>--lang zh en</code>
<code>npx opsx-dev-pipeline@latest upgrade</code>	更新已托管 Asset，并采纳当前 CLI 版本新增的模板	<code>--dry-run</code> 、 <code>--yes</code> 、 <code>--force</code> 、 <code>--lang zh en</code>
<code>npx opsx-dev-pipeline@latest uninstall</code>	删除 Manifest 跟踪的托管 Asset；可只卸载指定 Tool	<code>--dry-run</code> 、 <code>--yes</code> 、 <code>--tool claude</code>

查看所有命令或某个命令的完整参数：

```
1 npx opsx-dev-pipeline@latest --help
2 npx opsx-dev-pipeline@latest init --help
3 npx opsx-dev-pipeline@latest sync --help
```

`/opsx:sync` 与 `npx opsx-dev-pipeline@latest sync` 不是同一个操作：前者同步 OpenSpec change 的 Delta Specs，后者同步 CLI 托管的 Skill、Command、Schema 等 Asset。

10. 常见问题

10.1 提示找不到 OpenSpec 或版本过低

```
1 npm install -g @fission-ai/openspec@latest
2 openspec --version
```

确认版本不低于 1.6.0，并确保安装目录已加入 `PATH`。

10.2 目标目录不是 Git 仓库，或 Git 身份缺失

在项目根目录初始化 Git，并配置提交身份：

```
1 git init
2 git config user.name "Your Name"
3 git config user.email "you@example.com"
```

只在尚未初始化 Git 或尚未配置身份时执行对应命令。Pipeline Phase 0 会检查这些条件；身份缺失可能只显示警告，但 Phase 6 commit 前必须补齐。

10.3 使用 `--yes` 时提示必须指定 Stack

非交互模式必须显式传入：

```
1 --stack frontend
2 --stack backend
3 --stack fullstack
```

例如：

```
1 npx opsx-dev-pipeline@latest init --tool claude --stack backend --yes
```

10.4 初始化时出现已有文件冲突

- 默认交互模式：逐项选择覆盖、跳过或追加；
- `--yes`：跳过冲突文件；
- `--force`：覆盖冲突文件。

已有项目应先运行 `--dry-run` 并审阅差异。不要在不清楚内容的情况下使用 `--force`。

10.5 项目没有 Git remote

可以继续开发，并在归档后选择 `local-only`。需要推送时先配置 remote：

```
1 git remote add origin <repository-url>
2 git remote -v
```

10.6 初始化后看不到 `/opsx:*` 命令

确认 `.claude/commands/opsx/` 已生成，然后重新打开 Claude Code 项目会话。如果目录缺失，运行：

```
1 | npx opsx-dev-pipeline@latest doctor
```

并根据诊断结果处理初始化或版本问题。

10.7 `sync` 和 `upgrade` 有什么区别

```
1 | # 重新渲染 Manifest 中已经托管的 Asset
2 | npx opsx-dev-pipeline@latest sync --dry-run
3 |
4 | # 同步已有 Asset，并采纳新版本新增的模板
5 | npx opsx-dev-pipeline@latest upgrade --dry-run
```

先使用 `--dry-run` 查看影响，再执行正式命令。